

Programming Project: Task Scheduler

Instructions:

- The project requires completing tasks by May 08, 2025.
- Your goal is to implement a Task Scheduler as a prompt window that allows users to manage a list of tasks. The task list must be internally stored using a linked list(s).
- A task display in a list must be as follows

$$\begin{cases} "id. [] title (priority)" & \text{if task is incompleted} \\ "id. [X] title (priority)" & \text{if task is completed} \end{cases}$$

- An individual task display must be as follows

$$\begin{cases} "id. [] title (priority)\ndescription" & \text{if task is incompleted} \\ "id. [X] title (priority)\ndescription" & \text{if task is completed} \end{cases}$$

- The scheduler must support and have a prompt command for the following operations:
 - Add a task (at the beginning, end, or a specific position).
 - Remove a task (by ID or position).
 - Edit a task's content (not including ID).
 - Mark a task as complete/incomplete.
 - Reorder tasks (move up/down in the list).
 - Display all tasks.
 - Display a detailed individual task.
 - Display a list of prompt commands.
- You must use the *Node* class from 'Node.h'.
- Each task must have at least the following attributes:
 - Unique ID generated when the task is created
 - Title
 - Description
 - Priority level that is either Low, Medium, or High.
 - Completion status (complete/incomplete)
- All classes that display content must inherit *Object*.
- All classes must be defined in header files.
- The header files can only use the libraries *iostream*, *string*, *sstream*, *cctype*, *iomanip*, *cmath*, 'Object.h', 'Node.h'. and any header file you define.
- Your final submission should include:
 - the source codes with comments.
 - documentation file (.pdf or .md) that contains:
 - a list and description of all classes you implemented such that pseudocode is provided for each method.
 - a brief explanation of the overall program design.
 - instructions on how to run your program.
- Cheating of any kind is prohibited and will not be tolerated.
- Violating and/or failing to follow any rules will result in an automatic zero (0) for the project.

Hints & Suggestions

- You should implement the classes
 - Task – Represents a single task item.
 - TaskList – Custom linked list class that handles adding, removing, editing, and moving tasks.
 - TaskScheduler – Main class provides the prompt window and interacts with the task list.